Tema 2.2: Desarrollo de aplicaciones para móviles – Menús y vistas

Laura Sacristán Matesanz / David López Cuenca

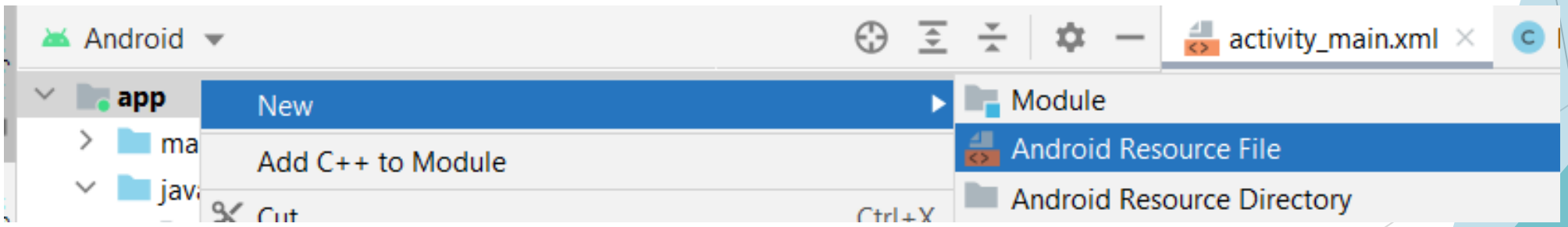
Menús

Menú I

- Los menús son elementos gráficos de una aplicación móvil que nos sirven para poder navegar por las distintas opciones que nos ofrezca nuestra App.
- Tenemos tres tipos diferentes de menús:
  - **Menú de opciones**: es la colección principal de elementos de menú de una actividad. Es donde debes colocar las acciones que tienen un impacto global en la app, como "Buscar", "Redactar correo electrónico" y "Configuración".
  - **Menú contextual**: es un menú flotante que aparece cuando el usuario hace un clic largo en un elemento. Proporciona acciones que afectan al contenido seleccionado o al marco contextual.
  - **Menú emergente:** muestra una lista de elementos en una lista vertical que está anclada a la vista que invocó el menú. Es adecuado para proporcionar una ampliación de acciones relacionadas con contenido específico o para proporcionar opciones en una segunda parte de un comando.

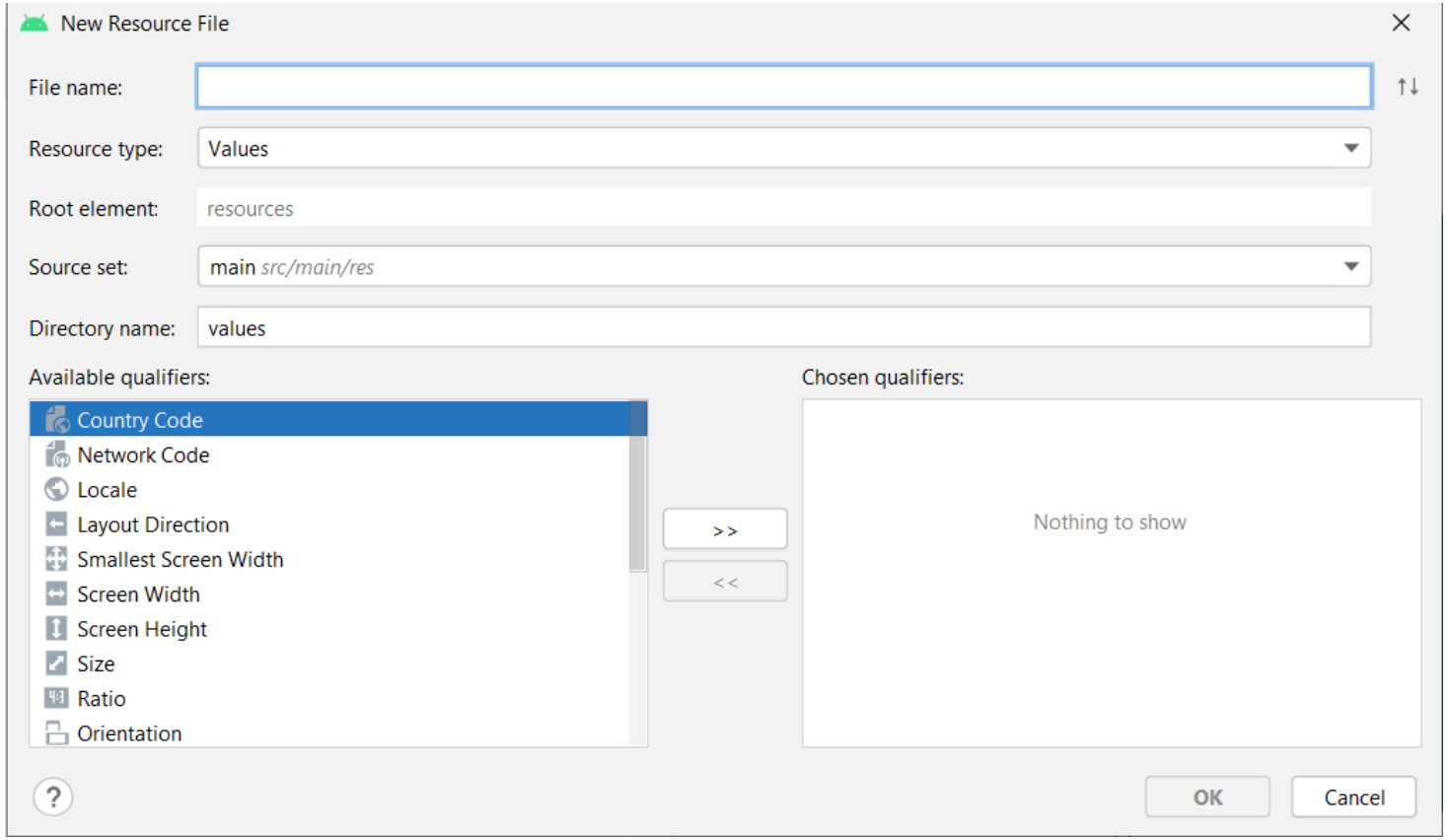
Menú II

- Vamos a crear un MENÚ DE OPCIONES. Para ello vamos a generar un nuevo proyecto, al que llamaremos CreacionMenu.
- Una vez creado, vamos a crear nuestro menú. Para ello vamos a pulsar el botón derecho sobre la app y elegiremos la opción new. Dentro de new vamos a elegir la opción Android Resource File. De esta manera crearemos un XML de tipo menú dentro de nuestra aplicación.



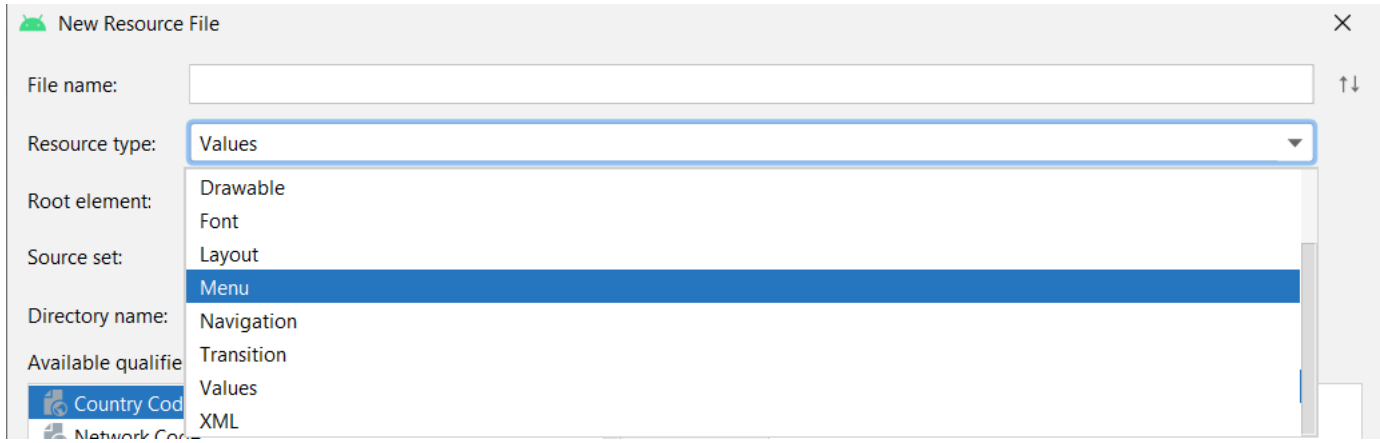
Menú III

- Una vez elegida la opción aparecerá la siguiente pantalla:



Menú IV

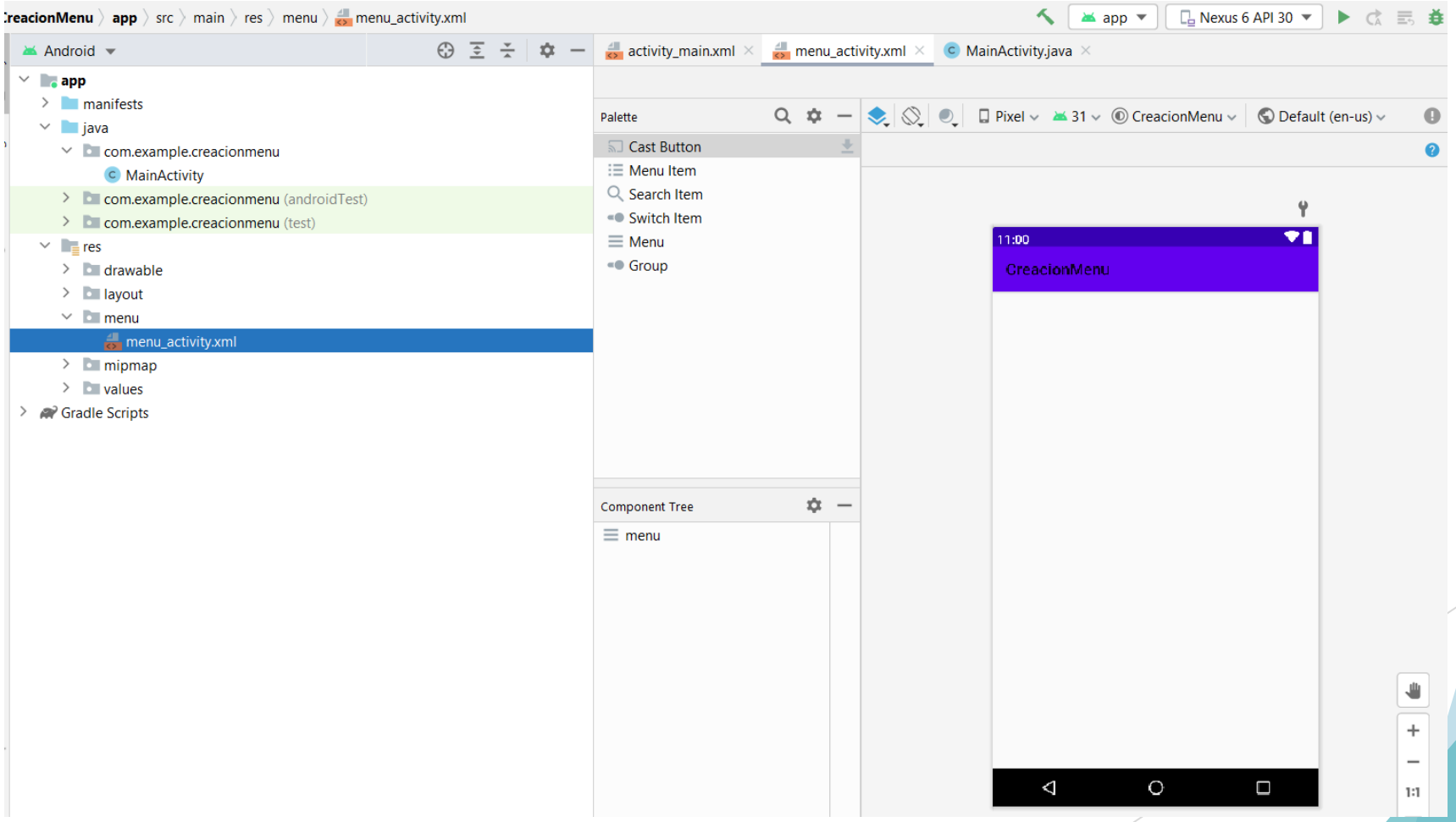
- Le daremos un nombre a nuestro menú en el campo File Name. En nuestro caso le llamaremos menu_activity.
- Dentro de la opción Resource type, le tenemos que indicar que tipo de recurso vamos a crear. Tenemos las siguientes opciones:



- Nosotros elegiremos Menu.

Menú V

- Se nos crea un fichero xml dentro de la carpeta res y en una subcarpeta llamada menu



Menú VI

- Puede darse el caso de que automáticamente Android Studio te genere la subcarpeta menú y dentro el archivo xml en el que cofiguraremos el menú.
- Esto se produce cuando creamos un proyecto nuevo y en lugar de elegir un empty_activity, elegimos por ejemplo un blank_activity. En ese caso creará directamente la opción de menú.
- Si observáis la pantalla anterior, podréis ver que en la paleta aparecen diferentes opciones de las que aparecen habitualmente.
- En este caso aparecen elementos referidos al menú, ya que estamos en un xml especifico para crearlo.
- Si accedemos a ver el fichero xml veremos lo siguiente:

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <menu xmlns:android="http://schemas.android.com/apk/res/android">
3
4  </menu>
```

Menú VII

- Para indicar que el menú lo vamos a mostrar en la actividad principal, tenemos que incluir en el XML unos parámetros que necesita la etiqueta menú.

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context=".">
                MainActivity (com.example.creacionmenu)
                Ctrl+Abajo and Ctrl+Arriba will move caret down and up in the editor Next Tip
</menu>
```

- En nuestro caso solo aparece la opción del MainActivity porque es la única actividad que tenemos. Si tuviéramos varias nos daría la opción de elegir.
- Con esto tenemos creado el menú, pero es un menú vacío.

Menú VIII

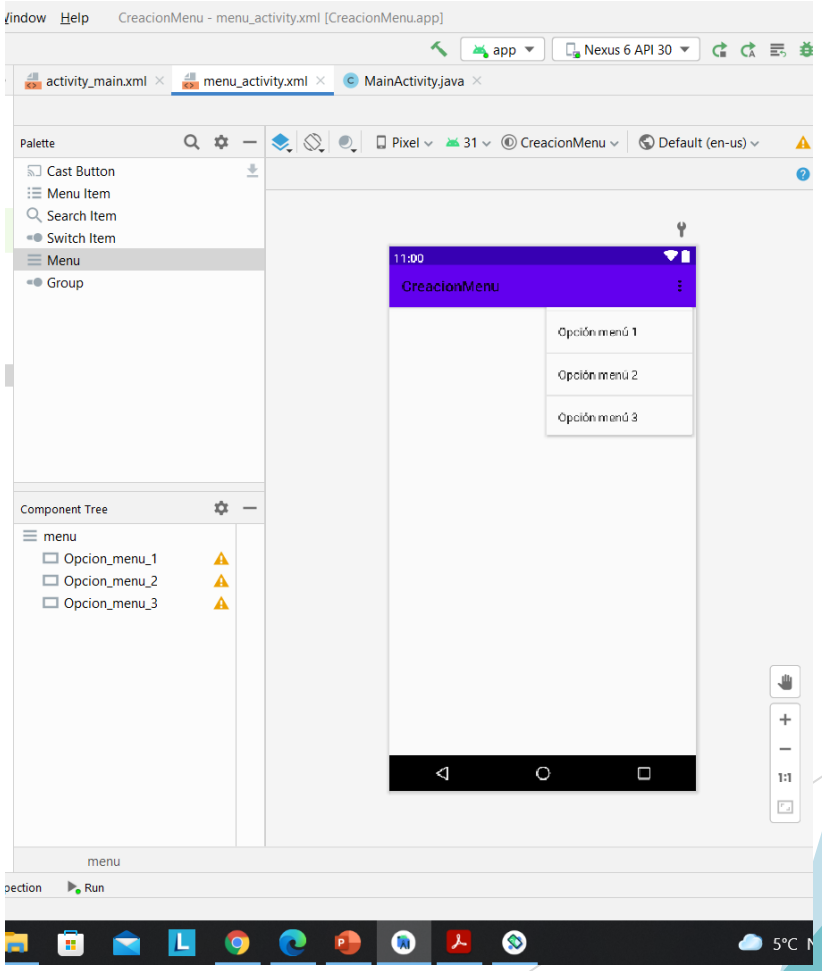
- El siguiente paso es confeccionar las opciones del menú.
- Para ello crearemos un elemento item por cada uno de los elementos que queremos que tenga el menú.

activity_main.xml | menu_activity.xml | MainActivity.java

```xml
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context=".MainActivity">
    <item android:id="@+id/Opcion_menu_1"
        android:title="Opción menú 1"/>
    <item android:id="@+id/Opcion_menu_2"
        android:title="Opción menú 2"/>
    <item android:id="@+id/Opcion_menu_3"
        android:title="Opción menú 3"/>

</menu>
```

Menú VIII

- Podemos añadir también todos los elementos a través del entorno gráfico. En la siguiente imagen podemos ver este entorno.



Menú IX

- El siguiente paso es activar el menú en el MainActivity.java.
- Para ello crearemos dos métodos:
  - onCreateOptionsMenu: para crear el menú propiamente dicho.
  - optionsItemSelected: para configurar las acciones.
- Estos métodos ya están creados dentro de la clase Activity los estamos heredando. Lo que vamos a hacer es sobrescribirlos.
- Para sobrescribirlo vamos a utilizar @Override y a continuación ponemos el primer método que vamos a crear.
- En este caso creamos onCreateOptionsMenu. Este método devuelve un valor booleano y como parámetro tenemos un menú.
- Dentro de este método debemos incluir otro método que va a activar el menú.
- Ese método es el getMenuInflater().inflate(); Tendremos que poner el id del menú y el nombre del menú.

Menú X

ity_main.xml | menu_activity.xml | MainActivity.java

```java
package com.example.creacionmenu;

import ...

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
    @Override
    public boolean onCreateOptionsMenu(Menu mimenu){
        getMenuInflater().inflate(R.menu.menu_activity,mimenu);
        return true;
    }
}
```

Menú XI

- El siguiente paso es indicar cual es la acción que realiza cada una de las opciones del menú.
- Para ello utilizaremos el método onOptionsItemSelected. Dentro de este método tendremos que capturar el valor del id del elemento del menú que seleccione el usuario.
- Una vez pulsada la opción del menú que se desee debemos de indicar en el programa que es lo que se debe hacer.
- El programa quedará de la siguiente manera:

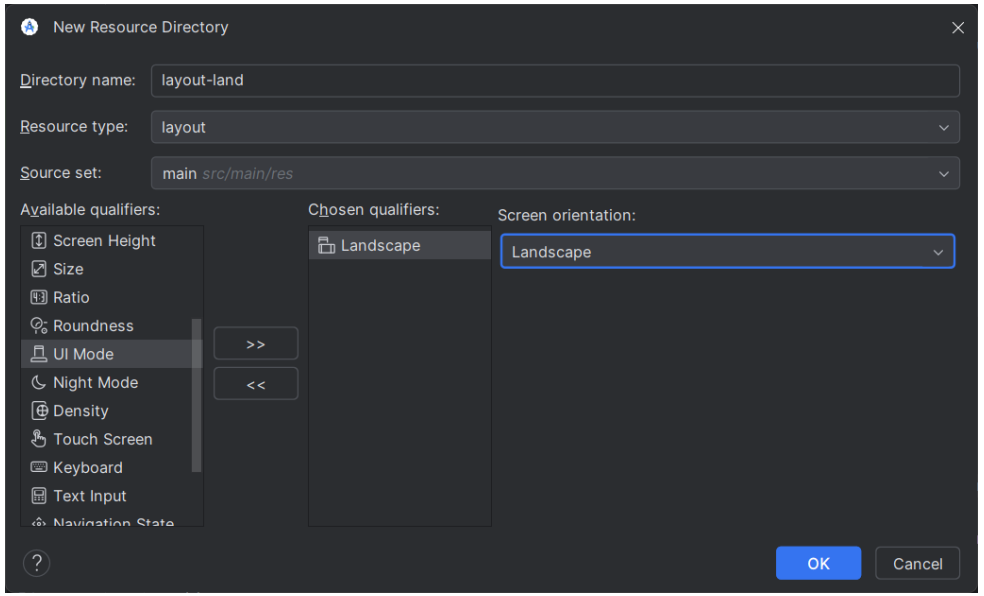
Menú XII

```
@Override
public boolean onOptionsItemSelected(MenuItem opcion_menu){
  int id=opcion_menu.getItemId();
  if (id==R.id.Opcion_menu_1){
      Toast.makeText( context: this,  text: "Opción menu 1", Toast.LENGTH_LONG).show();
      return true;
    }
    if (id==R.id.Opcion_menu_2){
        Toast.makeText( context: this,  text: "Opción menu 2", Toast.LENGTH_LONG).show();
        return true;
    }
    if (id==R.id.Opcion_menu_3){
        Toast.makeText( context: this,  text: "Opción menu 3", Toast.LENGTH_LONG).show();
        return true;
    }
    return super.onOptionsItemSelected(opcion_menu);
}
```

Vistas

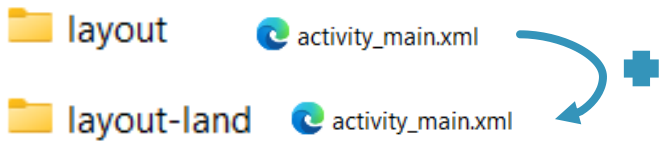
Vista horizontal vs vertical I

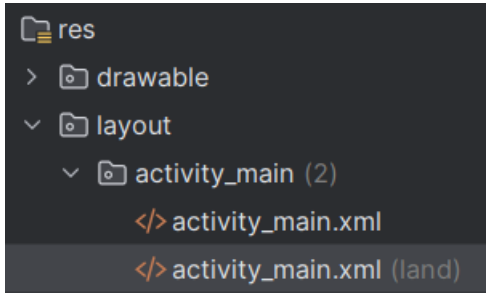
- Podemos crear una vista horizontal asociada a una vertical que se mostrará automáticamente cuando rotemos el dispositivo.
- Para ello se crea un nuevo directorio de recursos, de tipo layout, seleccionando el cualificador de Orientation y poniendo la orientación horizontal (Landscape).



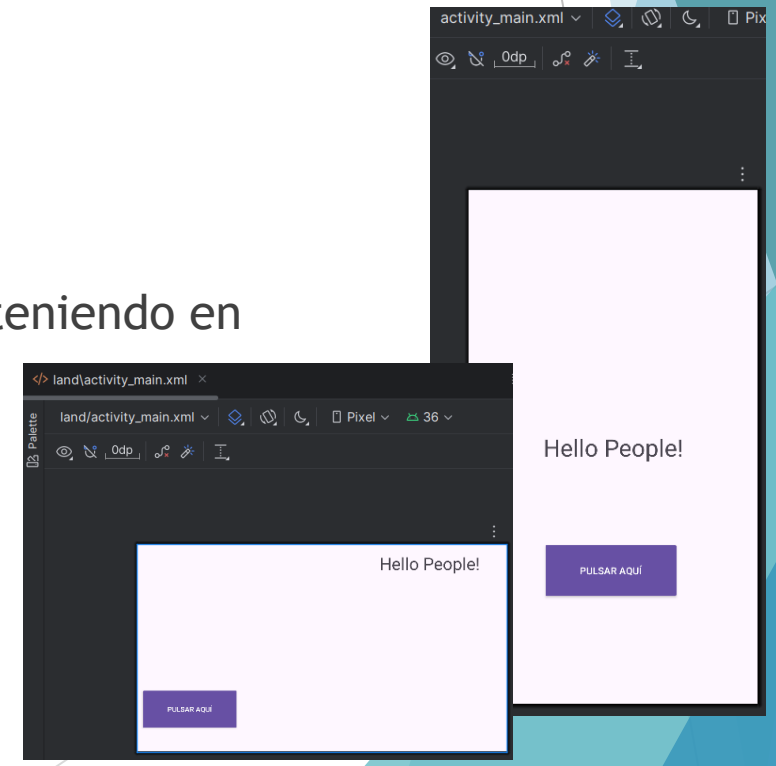
Vista horizontal vs vertical I

- Los componentes deberán replicarse en ambas por lo que podemos duplicar el layout original y moverlo a la nueva carpeta generada.





- Podemos modificar independientemente los componentes, teniendo en consideración la coherencia entre ambas vistas.



ListView I

- Vamos a crear un proyecto nuevo al que llamaremos ListViewSimple.
- De momento escogeremos una plantilla empty, aunque podríamos escoger una plantilla basic que contiene botón flotante y nos podría ayudar.
- Lo primero que vamos a hacer es crear un elemento ListView. Como siempre lo podemos hacer a nivel de código o en el entorno gráfico.

ListView II

- Si lo queremos hacer de manera gráfica, accederemos a la legacy de la paleta y elegiremos la opción ListView.
- En cualquiera de los dos casos se nos genera una caja dentro del activity_main que tendremos que ocupa toda la pantalla y que tendremos que referenciar a los márgenes, como cualquier elemento que incluimos en el layout.
- Nuestra lista está vacía y tendremos que rellenarla con el listado de datos que necesitemos.
- Para ello tendremos que programar en el MainActivity para mostrar el listado. Definimos una variable de tipo ListView y la referenciaremos a la lista generada en el activity_main.

ListView III

- Lo siguiente será generar el listado que queremos que aparezca en la ListView.
- Vamos a definir una variable de lista de strings que vamos a llamar ejemplosList.
- Para poder rellenar esa lista de ejemplos tendremos que crear un elemento Arraylist que nos dará la opción de ir poniendo todos los elementos que necesitemos en nuestra lista.
- El MainActivity quedará de la siguiente manera:

ListView IV

```java
package com.example.listviewsimple;

import ...

public class MainActivity extends AppCompatActivity {

    private ListView lista;
    private List<String> ejemploslist;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        lista=(ListView) findViewById(R.id.Listviewejemplo);
        ejemploslist = new ArrayList<>();
        ejemploslist.add("Ejemplo 1");
        ejemploslist.add("Ejemplo 2");
        ejemploslist.add("Ejemplo 3");
        ejemploslist.add("Ejemplo 4");

    }
}
```

ListView V

- Si probamos el código realizado hasta ahora en la aplicación, no aparecerá nada.
- Para que la lista aparezca necesitamos conectar el listview con los elementos string creados. Para realizar esta conexión necesitamos un compenente llamado **ADPATER.**
- No hay una forma natural de unir la lista de elementos con la listview y por eso necesitamos los adaptadores.
- Los adaptadores toman la lista de elementos, toman la listview y toman una plantilla, es decir, layout que actúa como diseño de la lista de elementos. Nos permitirá indicar de que manera queremos mostrar cada elemento.
- En Android existe por defecto un adaptador (ArrayAdapter) y un layout, que es el que vamos a utilizar en este caso.

ListView VI

- Vamos a crear un adaptador que llamaremos adaptadorEjemplo.
- Dentro del adaptador tendremos que indicar los parámetros que necesita:
  - El primero es el contexto, que en este caso será this.
  - El siguiente parámetro será el layout que voy a utilizar para dibujar cada elemento de la lista. Para ello utilizaremos un layout de la librería Android. El layout que elegiremos únicamente tiene un textView, por lo que solo podremos poner un texto en cada uno de los elementos.
  - Por último indicaremos el listado de elementos que queremos que nos muestre.
- El código quedará de la siguiente manera:

```java
public class MainActivity extends AppCompatActivity implements AdapterView.OnItemClickListener {

    private ListView lista;
    private List<String> ejemploslist;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        lista=(ListView) findViewById(R.id.Listviewejemplo);
        ejemploslist = new ArrayList<>();
        ejemploslist.add("");// pongo este elemento de la lista porque si no el nombre del activity t
        //apa el primer elemento de la lista.
        ejemploslist.add("Ejemplo 1");
        ejemploslist.add("Ejemplo 2");
        ejemploslist.add("Ejemplo 3");
        ejemploslist.add("Ejemplo 4");
        ArrayAdapter<String> adaptadorEjemplo = new ArrayAdapter<String>(
                context: this,
                android.R.layout.simple_list_item_1,
                ejemploslist
        );
        lista.setAdapter(adaptadorEjemplo);
        //Evento click sobre cada elemento de la lista.
        lista.setOnItemClickListener(this);
    }

    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
        Toast.makeText( context: this,  text: "El ejemplo seleccionado es: "+ejemploslist.get(position), Toast.LENGTH_SHORT).show();
    }
}
```

ListView VII

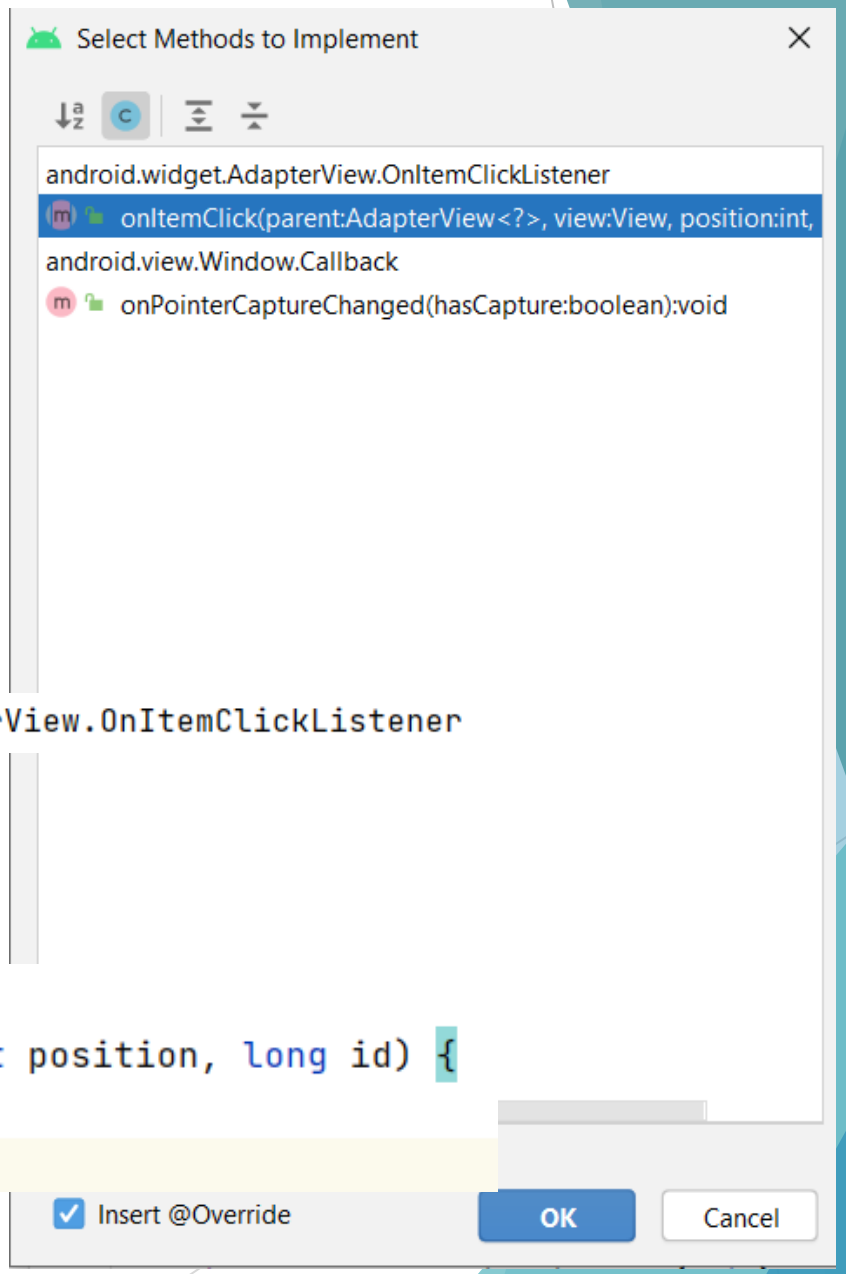
- Lo siguiente que tendremos que realizar es gestionar los eventos click de cada una de las opciones.
- Le vamos a indicar que la clase que va a implementar la interfaz es el propio MainActivity por lo que pondremos this. Nos va a devolver un error, porque el propio programa detecta que no puede resolverlo.
- Elegiremos la opción de hacer que el MainActivity implemente la interfaz. Para ello pulsaremos las teclas Alt+enter encima de this y elegiremos la segunda opción que nos aparece.

```
        //Evento click sobre cada elemento de la lista.
        lista.setOnItemClickListener(this);
    }
}
```

Cast parameter to 'android.widget.AdapterView.OnItemClickListener'
Make 'MainActivity' implement 'android.widget.AdapterView.OnItemClickListener'
Press Ctrl+Mayús+I to open preview

ListView VIII

- Al pulsar la opción aparece la siguiente pantalla.
- Nos obliga a sobreescribir el método onItemClick.
- Y esto conlleva que se mofique el MainActivity en su cabecera para que pueda implementar el método onItemClickListener




```
public class MainActivity extends AppCompatActivity implements AdapterView.OnItemClickListener
```

- Y nos aparece el método

```
@Override
public void onItemClick(AdapterView<?> parent, View view, int position, long id) {

}
```

ListView IX

- Este método va a ser lanzado cada vez que pulsemos uno de los elementos de la lista.
- Para que pueda identificar cual elemento hemos pulsado, al parámetro int i del método le vamos a cambiar el nombre y vamos a llamarlo posición.
- Como hemos creado un array, el primer elemento de la lista tiene la posición 0, el siguiente la posición 1 y así consecutivamente. Esas mismas posiciones están en nuestra lista.
- Como tenemos definida la lista a nivel de clase, vamos a poder acceder a ella desde el método sin problema.
- Para ver que accedemos a cada uno de los elementos vamos a poner un toast para visualizarlo.

```java
public class MainActivity extends AppCompatActivity implements AdapterView.OnItemClickListener {

    private ListView lista;
    private List<String> ejemploslist;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        lista=(ListView) findViewById(R.id.Listviewejemplo);
        ejemploslist = new ArrayList<>();
        ejemploslist.add("");// pongo este elemento de la lista porque si no el nombre del activity t
        //apa el primer elemento de la lista.
        ejemploslist.add("Ejemplo 1");
        ejemploslist.add("Ejemplo 2");
        ejemploslist.add("Ejemplo 3");
        ejemploslist.add("Ejemplo 4");
        ArrayAdapter<String> adaptadorEjemplo = new ArrayAdapter<String>(
                context: this,
                android.R.layout.simple_list_item_1,
                ejemploslist
        );
        lista.setAdapter(adaptadorEjemplo);
        //Evento click sobre cada elemento de la lista.
        lista.setOnItemClickListener(this);
    }

    @Override
    public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
        Toast.makeText( context: this,  text: "El ejemplo seleccionado es: "+ejemploslist.get(position), Toast.LENGTH_SHORT).show();
    }
}
```

Ejemplo menú

AHORA OS TOCA TRABAJAR A VOSOTROS.

DEBEIS CREAR UN MENU DE OPCIONES EN EL QUE APAREZCAN 3 COMUNIDADES AUTÓNOMAS. P.E. CATALUÑA, MADRID Y ARAGÓN

CUANDO SE PULSE ALGUNA DE LAS COMUNIDADES DEBERÁN MOSTRARSE SUS PROVINCIAS EN UN LISTVIEW DE LA APLICACIÓN.

CUANDO HAYAS FINALIZADO AVISA AL PROFESOR PARA REVISARLO.

ListView personalizado

- Hasta ahora hemos realizado un ListView simple utilizando un layout existente, pero si queremos personalizar nuestro propio listado, tendremos que diseñar nuestro propio layout para nuestra ListView.
- Para ello vamos a generar un proyecto nuevo que llamaremos listview_personalizado.
- Vamos a crear un elemento listview en el archivo activity_main.xml como hemos hecho en el ejemplo anterior.
- También modificaremos el MainActivity definiendo una variable de tipo listview, como hicimos en el ejemplo anterior.
- En principio los archivos quedarán así:

ListView personalizado

```xml
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <ListView
        android:id="@+id/listViewEjemplo"
        android:layout_width="398dp"
        android:layout_height="713dp"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

```java
package com.example.listviewpersonalizado;

import ...

public class MainActivity extends AppCompatActivity {
    ListView lista;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        lista=(ListView) findViewById(R.id.listaViewEjemplo);
    }
}
```

ListView personalizado

- Lo que hemos realizado en el ejercicio anterior, es generar un listado de elementos de tipo TextView, pero en este caso vamos a realizar un listado de elementos que tendrá una imagen o ruta url a un servidor donde esté la imagen, varios string y un número.
- En realidad podremos personalizarlo de la manera que necesitemos visualizar el listado de elementos.
- Para poder generar esta lista tendremos que definir una lista del tipo ejemplo que llamaremos ejemplo list. Esto lo añadiremos en nuestro ActivityMain.

```
list<Ejemplo> ejemplolist;
```

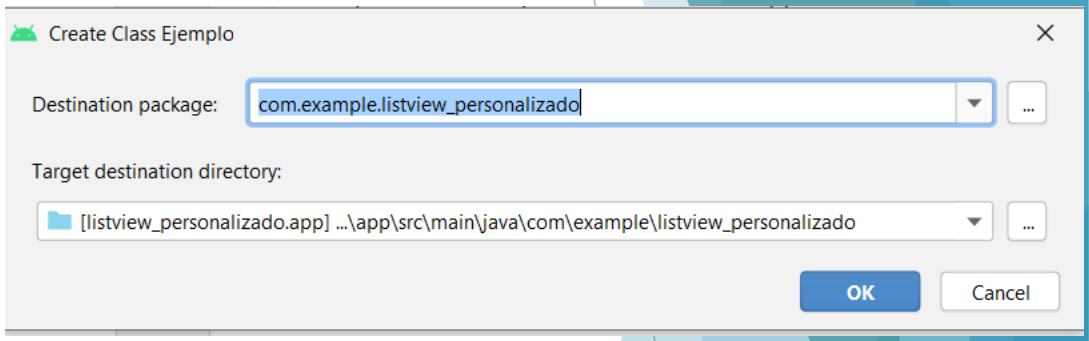
- Veremos que Ejemplo aparece como error. Eso es debido a que Ejemplo debe ser una clase que aún no existe, por lo que deberemos crearla.

ListView personalizado

```
package com.example.listview_personalizado;

import ...

public class MainActivity extends AppCompatActivity {
    private ListView lista;
    list<Ejemplo> ejemplolist;

    @Overrid
    protecte                                    dInstanceState) {
        supe                                    te);
        setC                                    y_main);
        list                                    (R.id.listViewEjemplo);
    }
}
```

Create class 'Ejemplo'
Create enum 'Ejemplo'
Create inner class 'Ejemplo'
Create interface 'Ejemplo'
Create type parameter 'Ejemplo'
Change access modifier
Press Ctrl+Mayús+I to open preview

Crearemos la clase Ejemplo



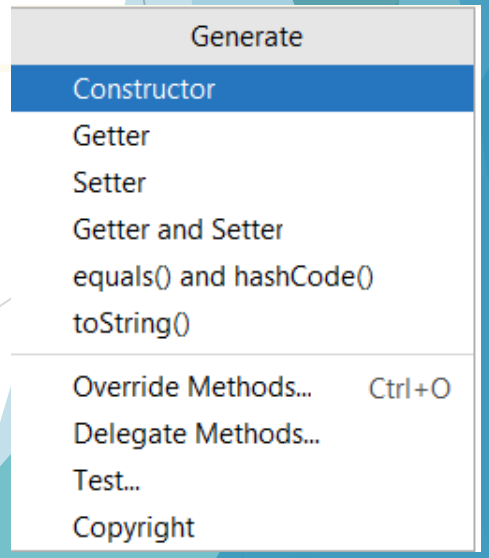
Creamos una clase java llamada ejemplo en la que definiremos las características que va a tener nuestro elemento de la vista.

ListView personalizado

- Un ejemplo podría ser:

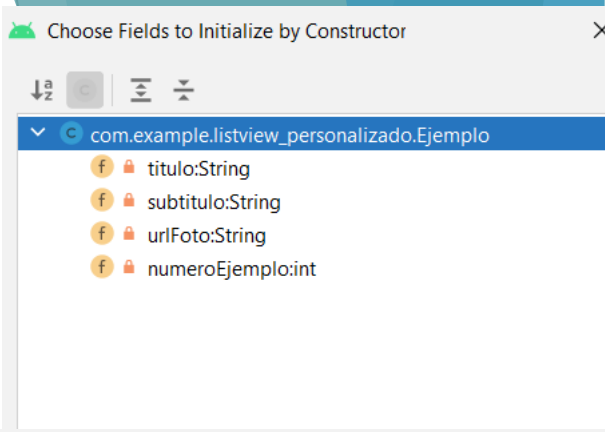
```
package com.example.listview_personalizado;

public class Ejemplo {
    private String titulo;
    private String subtitulo;
    private String urlFoto;
    private int numeroEjemplo;
}
```

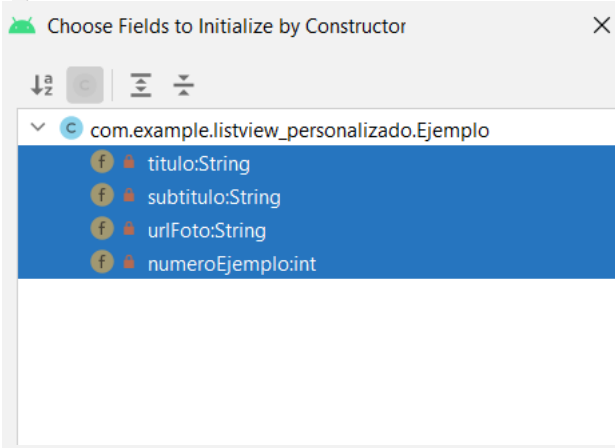
- Ahora vamos a generar el método constructor. Es una opción que nos facilita Android Studio. Para ello accederemos al menú code y dentro de este generate y elegimos la opción constructor

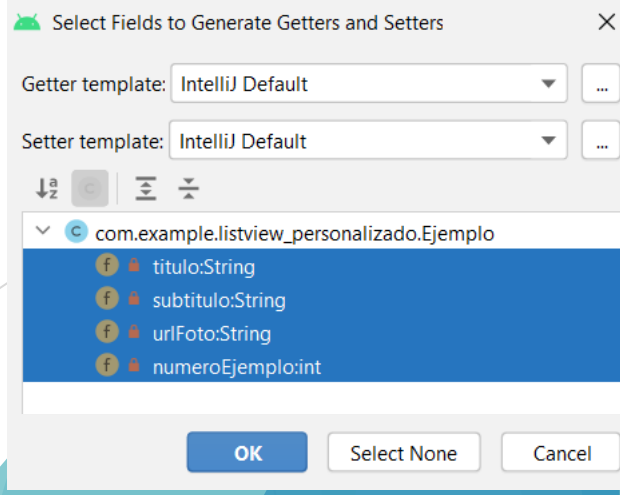


ListView personalizado

- Vamos a dejar un constructor en blanco y para ello elegimos la primera opción.
- También vamos a generar otro constructor en el que recibimos todos los parámetros que tenemos como atributos en nuestra clase. Para ello volvemos a code\generate y seleccionamos todos los atributos y damos a OK.
- Por último nos queda generar todos los métodos para cambiar u obtener los valores de estas propiedades. Para ello volvemos a code\generate y esta vez elegimos getter and setter. Elegimos de nuevo todos los atributos y le damos a ok.
- El resultado en el fichero Ejemplo.java será el siguiente:







```java
package com.example.listviewpersonalizado;

public class Ejemplo {
    private String titulo;
    private String subtitulo;
    private String urlFoto;
    private int numeroEjemplo;

    public Ejemplo() {
    }

    public Ejemplo(String titulo, String subtitulo, String urlFoto, int numeroEjemplo) {
        this.titulo = titulo;
        this.subtitulo = subtitulo;
        this.urlFoto = urlFoto;
        this.numeroEjemplo = numeroEjemplo;
    }
```

```java
public String getTitulo() {
    return titulo;
}

public void setTitulo(String titulo) {
    this.titulo = titulo;
}

public String getSubtitulo() {
    return subtitulo;
}

public void setSubtitulo(String subtitulo) {
    this.subtitulo = subtitulo;
}

public String getUrlFoto() {
    return urlFoto;
}

public void setUrlFoto(String urlFoto) {
    this.urlFoto = urlFoto;
}

public int getNumeroEjemplo() {
    return numeroEjemplo;
}

public void setNumeroEjemplo(int numeroEjemplo) {
    this.numeroEjemplo = numeroEjemplo;
}
```

ListView personalizado

- Volvemos al MainActivity e importamos la clase list para poder diseñar nuestro listado de elementos.
- Lo siguiente será generar nuestra lista de elementos y lo que haremos será crear un nuevo arraylist y después ir rellenando los elementos. Las líneas de código que debemos incluir son:

```
ejemploList = new ArrayList<>();
ejemploList.add(new Ejemplo("Título Ejemplo 1","Subtitulo Ejemplo 1","",2));
```

- Podemos incluir varios add para que nos salgan varios elementos y ver como funciona.
- El siguiente paso es incluir el adapatador. En este caso vamos a tener que generar un adaptador propio que tampoco existe, por lo que para generarlo tenemos que crear una clase que defina el adaptador.

```
MiAdapatadorEjemplo adapatadorEjemplo = new MiAdaptadorEjemplo;
```

ListView personalizado

- Creamos la clase adaptador como lo hemos hecho anteriormente, pero en este caso la clase va a extender de la clase ArrayAdapter.

```
public class MiAdapatadorEjemplo extends ArrayAdapter<Ejemplo> {
}
```

- Nos aparece como error porque necesitamos que exista un constructor. El constructor lo crearemos como lo hemos hecho en la clase anterior (code\generate\constructor)
- Nos van a aparecer diferentes opciones pero nosotros elegiremos el constructor que tiene por parámetros el contexto, un layout para dibujar cada elemento de la lista y la lista de elementos propiamente dicha. El constructor será este:

```
ArrayAdapter(context:Context, resource:int, objects:List<T>)
```

ListView personalizado

- La clase programada quedará de la siguiente manera:

```
package com.example.listview_personalizado;

import android.content.Context;
import android.widget.ArrayAdapter;

import androidx.annotation.NonNull;

import java.util.List;

public class MiAdaptadorEjemplo extends ArrayAdapter<Ejemplo> {
    public MiAdaptadorEjemplo(@NonNull Context context, int resource, @NonNull List<Ejemplo> objects) {
        super(context, resource, objects);
    }
}
```

ity_main.xml | MainActivity.java | MiAdapatardorEjemplo.java | Ejemplo.java

```java
import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.widget.ListView;

import java.util.ArrayList;
import java.util.List;

public class MainActivity extends AppCompatActivity {
    ListView lista;
    List<Ejemplo> ejemploList;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        lista = (ListView) findViewById(R.id.listViewEjemplo);
        ejemploList = new ArrayList<>();
        ejemploList.add(new Ejemplo( titulo: "Tituto Ejemplo 1",  subtitulo: "Subtitulo Ejemplo 1", urlFoto: "", numeroEjemplo: 1));
        ejemploList.add(new Ejemplo( titulo: "Tituto Ejemplo 2",  subtitulo: "Subtitulo Ejemplo 2", urlFoto: "", numeroEjemplo: 2));
        ejemploList.add(new Ejemplo( titulo: "Tituto Ejemplo 3",  subtitulo: "Subtitulo Ejemplo 3", urlFoto: "", numeroEjemplo: 3));
        ejemploList.add(new Ejemplo( titulo: "Tituto Ejemplo 4",  subtitulo: "Subtitulo Ejemplo 4", urlFoto: "", numeroEjemplo: 4));
```

ListView personalizado

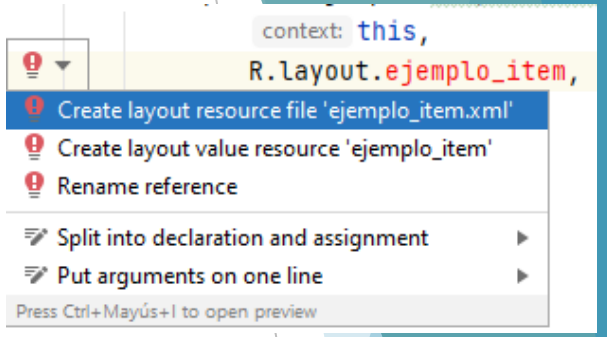
- Volvemos al MainActivity y en el momento en el que estamos creando el adaptador, le tendremos que pasar los parámetros. El código sería:

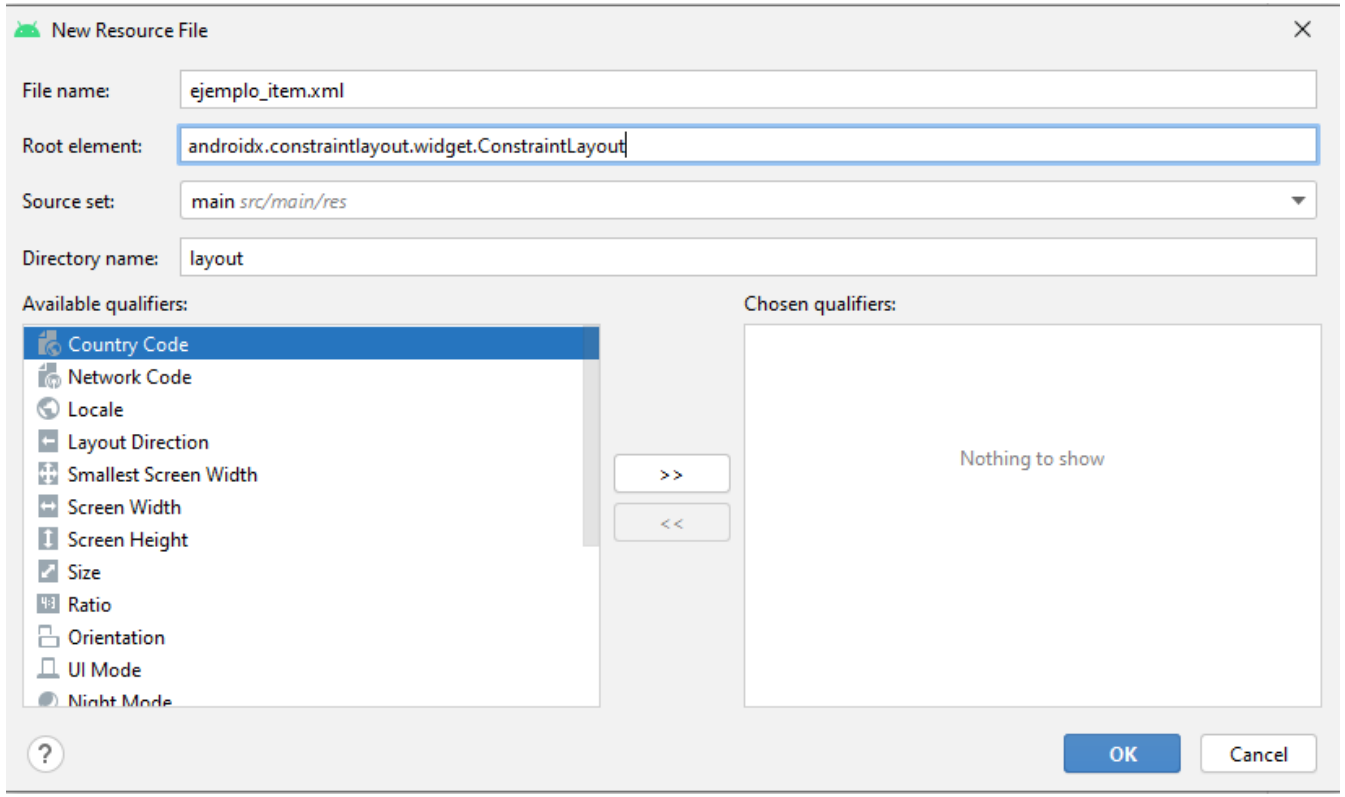
```
MiAdaptadorEjemplo adaptadorEjemplo = new MiAdaptadorEjemplo(
        this,
        R.layout.ejemplo_item,
        ejemploList
);
```

- Cuando pasamos el layout, nos va a dar un error porque no lo tenemos creado.
- Para solucionar el problema tendremos que crear el fichero de layout.

ListView personalizado

- En este layout vamos a diseñar un elemento de la lista para mostrarlo.





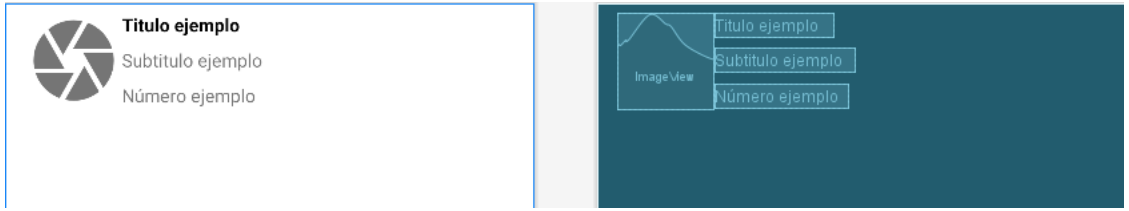
- Le damos a OK y nos genera el layout de diseño del elemento de la lista.

ListView personalizado

- Vamos a añadir entonces a este layout todos los elementos que necesitamos:
  - Imagen
  - 2 TextView
  - Número
- Para la imagen vamos a crear un nuevo Vector Asset y vamos a elegir una imagen de las que vienen por defecto en Android Studio.
- En el layout incluiremos un ImageView al que le asignaremos la imagen anterior. Su id será imageViewFoto.
- Después incluiremos un TextView. Su id será textViewTitulo. Lo uniremos por arriba y por el centro con la imagen. El texto estará en negro y negrita.
- Añadimos otro TextView. Su id será textViewSubtitulo y lo uniremos por arriba con la parte de abajo del otro TextView y también alineado por la izquierda.
- Por último nos falta otro TextView (en el que introduciremos el número). Estará unido por arriba con el TextView anterior y también alineado por la izquierda.

ListView personalizado

- El resultado es:



- Lo siguiente que tenemos que realizar es el diseño de nuestro adaptador.
- Para ello, dentro de la clase MiAdaptadorEjemplo, vamos a definir una serie de variables que vamos a necesitar.
  - Context ctx;
  - int layoutTemplate;
  - List<Ejemplo> ejemploList;
- Son los tres parámetros que necesita nuestro método MiAdaptadorEjemplo.

ListView personalizado

- Lo siguiente que vamos a hacer es ir guardando la información a medida que vamos recibiendo los parámetros en el adaptador. Utilizaremos el siguiente código:

```
this.ctx = context;
this.layoutTemplate = resource;
this.ejemploList = objects;
```

- Ahora vamos a sobreescribir el método getView (se genera de manera automática). Se lanza automáticamente como si fuera un bucle, una vez por cada elemento que recibimos en el adaptador. Es decir que como recibimos tres elementos se ejecutará tres veces.
- Cada vez que lo invocamos se va a recibir por parámetro la posición de la lista de elementos que está recorriendo en ese momento. Se repetirá hasta el último elemento del array que nos estén pasando por parámetro.

ListView personalizado

- El código que tenemos hasta el momento en la clase MiAdatadorEjemplo es:

```java
package com.example.listview_personalizado;

import ...

public class MiAdaptadorEjemplo extends ArrayAdapter<Ejemplo> {
    //Definición de variables.
    Context ctx;
    int layoutTemplate;
    List<Ejemplo> ejemploList;

    public MiAdaptadorEjemplo(@NonNull Context context, int resource, @NonNull List<Ejemplo> objects) {
        super(context, resource, objects);
        //A medida que recibo los parámetros, vamos a guardar las variables.
        this.ctx = context;
        this.layoutTemplate = resource;
        this.ejemploList = objects;

    }

    @NonNull
    @Override
    public View getView(int position, @Nullable View convertView, @NonNull ViewGroup parent) {
        return super.getView(position, convertView, parent);
    }
}
```

ListView personalizado

- A continuación vamos a borrar la línea de código que tenemos dentro del getView, para hacer nuestra propia programación.
- El código que incluiremos nos va a permitir obtener el layout y cargarlo dentro del elemento padre que es el listView que hemos recibido a través del activity.
- De esta manera, a medida que vaya recorriendo los elemento de la lista, los irá incluyendo en el listView.
- La variable v tendrá en este momento el layout en el que hemos definido el elemento de la lista.
- Al final de las operaciones debemos devolverlo para que se incluya en el listView como un elemento hijo.

```java
package com.example.listview_personalizado;

import ...

public class MiAdaptadorEjemplo extends ArrayAdapter<Ejemplo> {
    //Definición de variables.
    Context ctx;
    int layoutTemplate;
    List<Ejemplo> ejemploList;

    public MiAdaptadorEjemplo(@NonNull Context context, int resource, @NonNull List<Ejemplo> objects) {
        super(context, resource, objects);
        //A medida que recibo los parámetros, vamos a guardar las variables.
        this.ctx = context;
        this.layoutTemplate = resource;
        this.ejemploList = objects;

    }

    @NonNull
    @Override
    public View getView(int position, @Nullable View convertView, @NonNull ViewGroup parent) {
        View v = LayoutInflater.from(ctx).inflate(layoutTemplate,parent, attachToRoot: false);

        return v;
    }
}
```

ListView personalizado

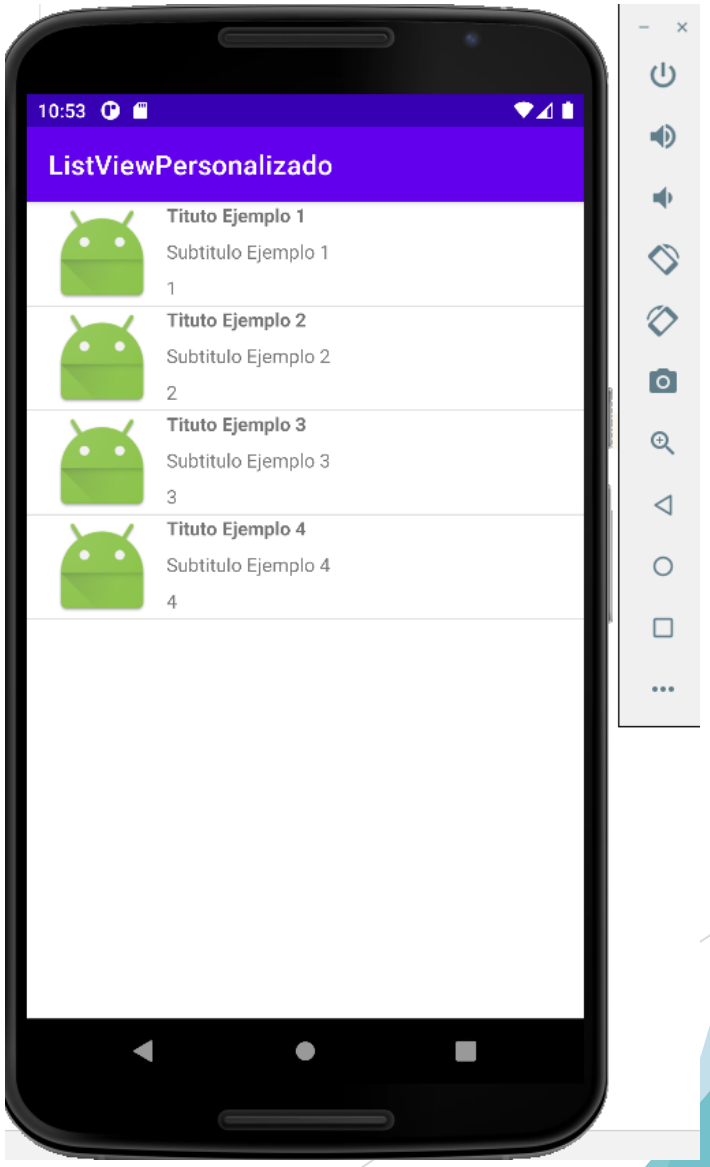
- Nos quedan varias operaciones que realizar antes de devolver la variable v.
- Lo primero es obtener la información del elemento de la lista que estamos recorriendo en este momento. Para ello pondremos el siguiente código:
  - Ejemplo elementoActual = ejemploList.get(position);
- A continuación vamos a rescatar los elementos de la interfaz de usuario de la tempate (plantilla). La plantilla está cargada en la variable v, por lo que el código que necesitaremos será:
  - TextView textViewTitulo = (TextView) v.findViewById(R.id.textViewTitulo);
  - TextView textViewSubtitulo = (TextView) v.findViewById(R.id.textViewSubtitulo);
  - TextView textViewNumero = (TextView) v.findViewById(R.id.textViewNumero);
- De esta manera ya tenemos los elementos de la vista.

ListView personalizado

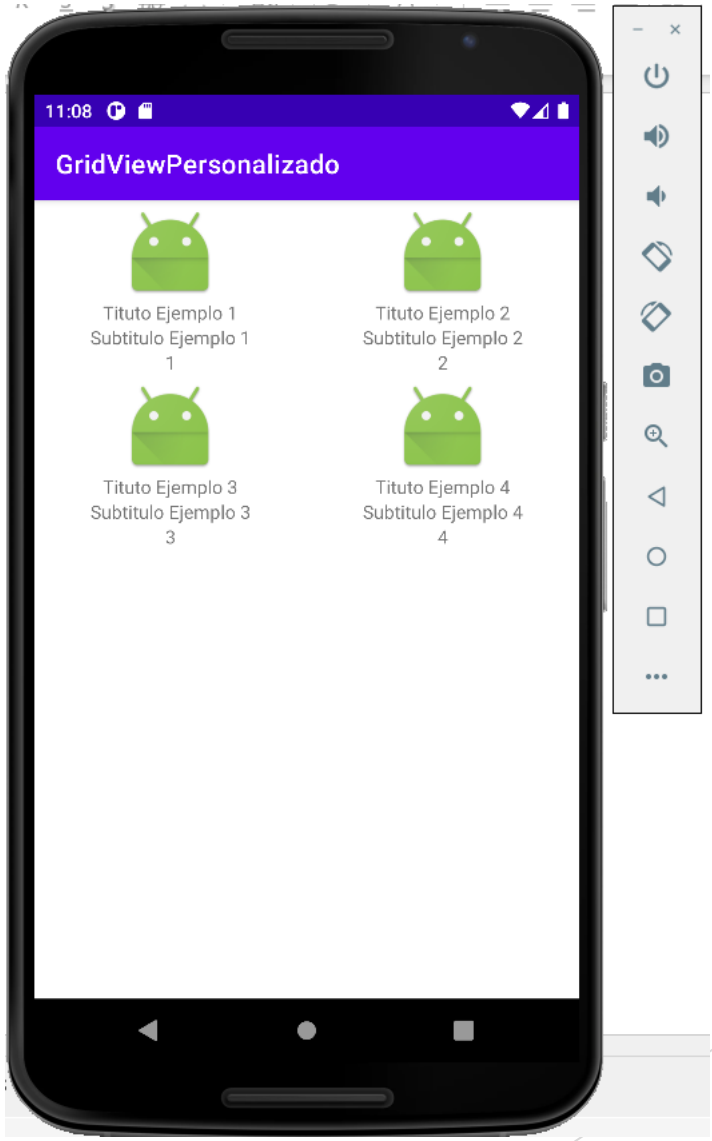
- Lo siguiente es hacer un set de la información del elemento Actual en los elementos de la interface de usuario. Para ello haremos las siguientes líneas de programación:
  - textViewTitulo.setText(elementoActual.getTitulo());
  - textViewSubtitulo.setText(elementoActual.getSubtitulo());
  - textViewNumero.setText(elementoActual.getNumeroEjemplo());
- El resultado final de la clase programada será el siguiente:

```java
public class MiAdapatardorEjemplo extends ArrayAdapter<Ejemplo> {
    //Definición de variables
    Context ctx;
    int layoutTemplate;
    List<Ejemplo> ejemploList;

    public MiAdapatardorEjemplo(@NonNull Context context, int resource, @NonNull List<Ejemplo> objects) {
        super(context, resource, objects);
        // A medida que recibo los parámetros, vamos a guardar las variables.
        this.ctx = context;
        this.layoutTemplate = resource;
        this.ejemploList = objects;
    }
    @NonNull
    @Override
    public View getView(int position, @Nullable View convertView, @NonNull ViewGroup parent) {
        View v = LayoutInflater.from(ctx).inflate(layoutTemplate,parent, attachToRoot: false);
        //Obtener la información del elemento de la lista que estamos recorriendo en este momento.
        Ejemplo elementoActual = ejemploList.get(position);
        //Rescatar los elementos de la interfaz de usuario de la template (plantilla)
        TextView textViewTitulo = (TextView) v.findViewById(R.id.textViewTitulo);
        TextView textViewSubtitulo = (TextView) v.findViewById(R.id.textViewSubtitulo);
        TextView textViewNumero = (TextView) v.findViewById(R.id.textViewNumero);
        //Hacer un set de la información del elementoActual en los elementos de la interfaz del usuario.
        textViewTitulo.setText(elementoActual.getTitulo());
        textViewSubtitulo.setText(elementoActual.getSubtitulo());
        textViewNumero.setText(elementoActual.getNumeroEjemplo()+"");

        return v;
    }
}
```

ListView personalizado



El resultado será el siguiente:



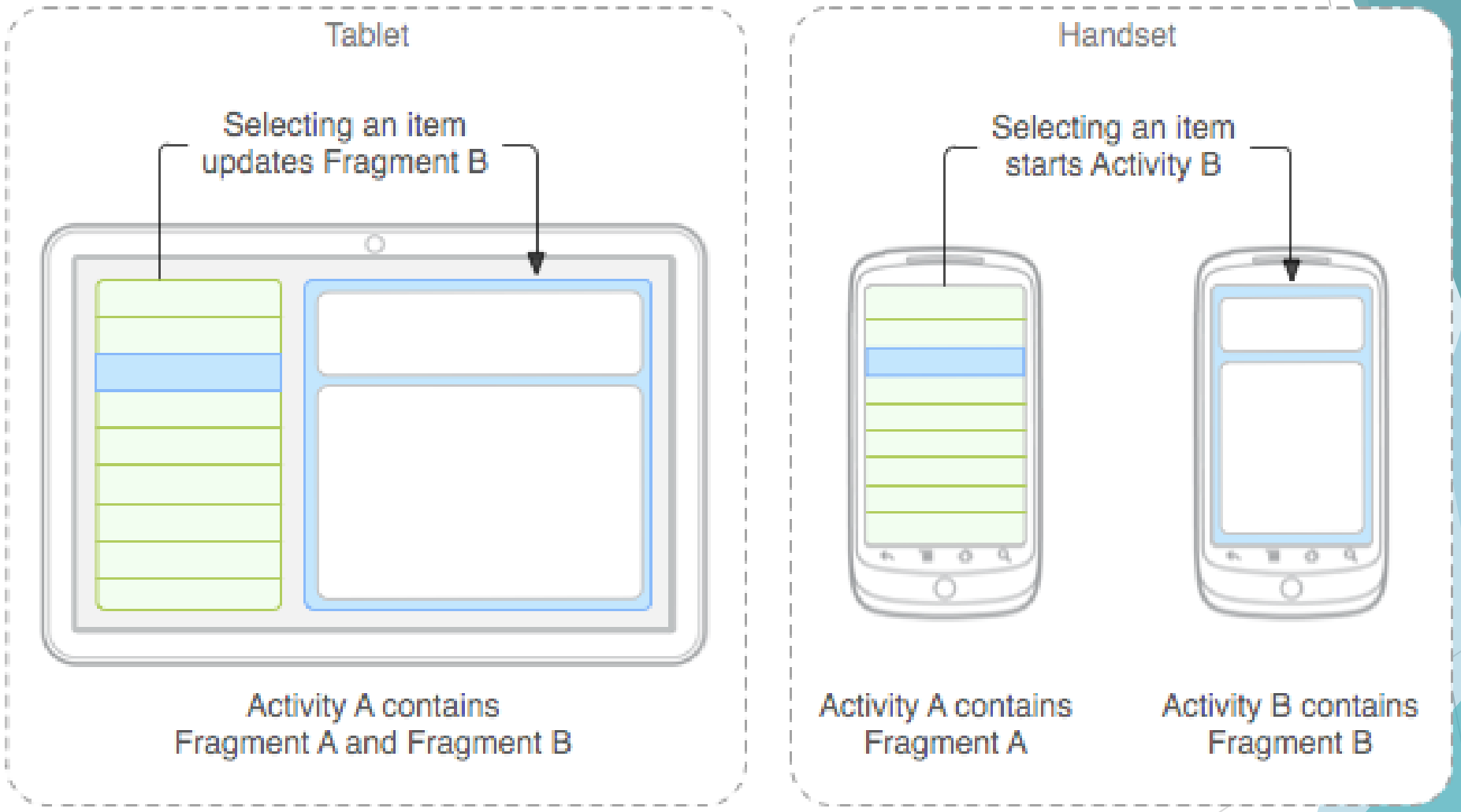
RecyclerView

- Otro elemento para generar listas el RecyclerView. **Es el más avanzado de todos y nos permite hacer listas con muchísimo elementos**.
- Con listview no podemos poner mucha cantidad de elementos porque la app se satura y da un error y se para.
- RecyclerView por lo tanto, va a ser más eficiente y más rápido.
- Para trabajar con RecycleView vamos a trabajar con fragmentos de tipo lista.
- Un fragmento es una parte de la interfaz de usuario. Se puede utilizar varios fragmentos para generar una actividad.

Los framentos permiten tener componentes de UI que se puedan poner en distintas actividades sin duplicar código.
Ejemplo:

- *Un Fragment que muestra una lista*
- *Un Fragment que muestra detalles*
- *Un Fragment que muestra un mapa*

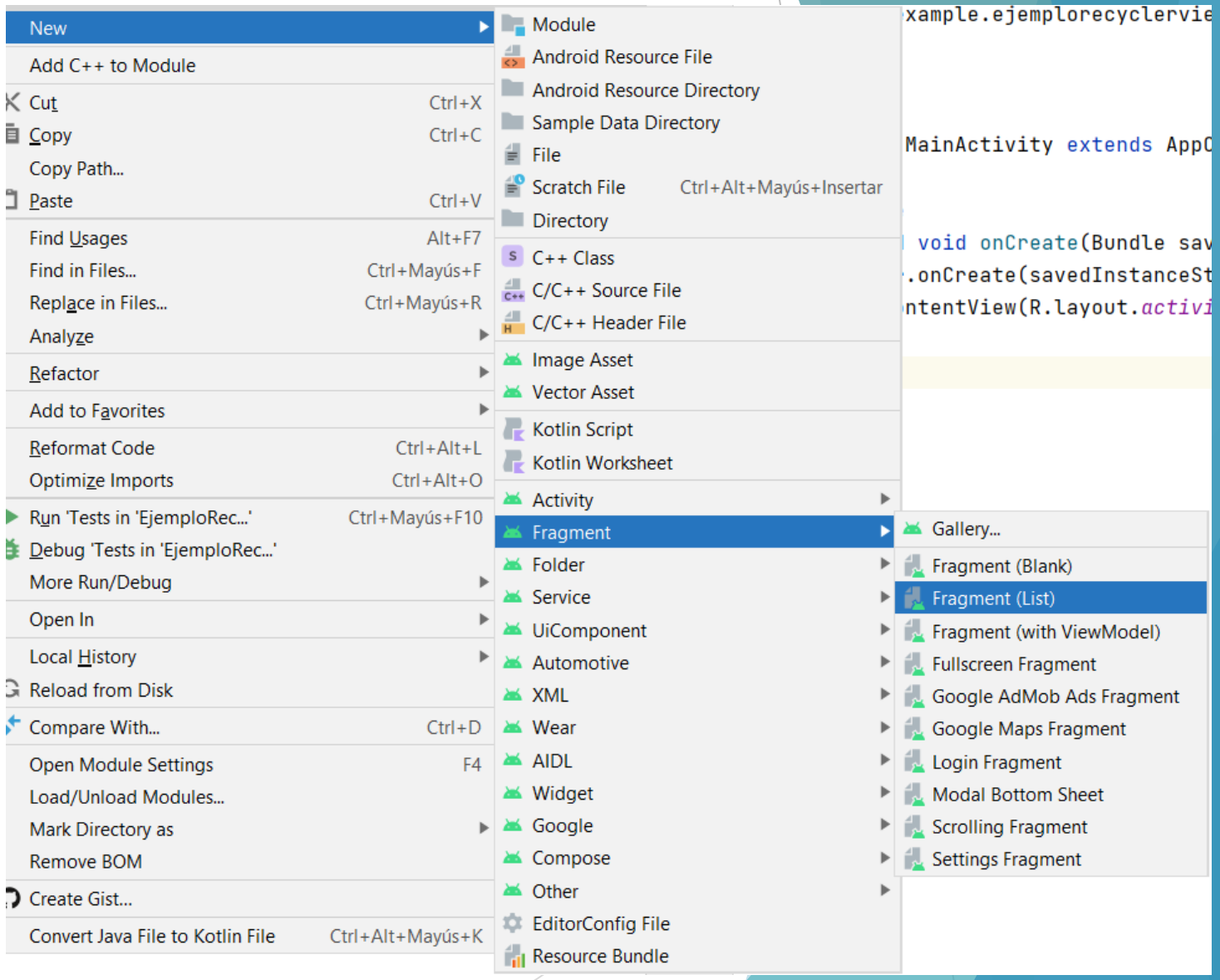
Son reutilizables en distintas pantallas.


Tablet
Selecting an item
updates Fragment B
Activity A contains
Fragment A and Fragment B
Handset
Selecting an item
starts Activity B
Activity A contains
Fragment A
Activity B contains
Fragment B

RecyclerView

Para generar un fragmento de tipo lista pulsaremos el botón derecho del ratón New\Fragment\Fragment(list).

Le pondremos como nombre Ejemplo, el resto de datos los dejaremos igual y pulsamos finalizar.



New Android Component

Object Kind

Ejemplo

Fragment class name

EjemploFragment

Column Count

1 (List)

Object content layout file name

fragment_item

List layout file name

fragment_item_list

Adapter class name

MyEjemploRecyclerViewAdapter

Source Language

Java

Target Source Set

main

Previous | Next | Cancel | Finish